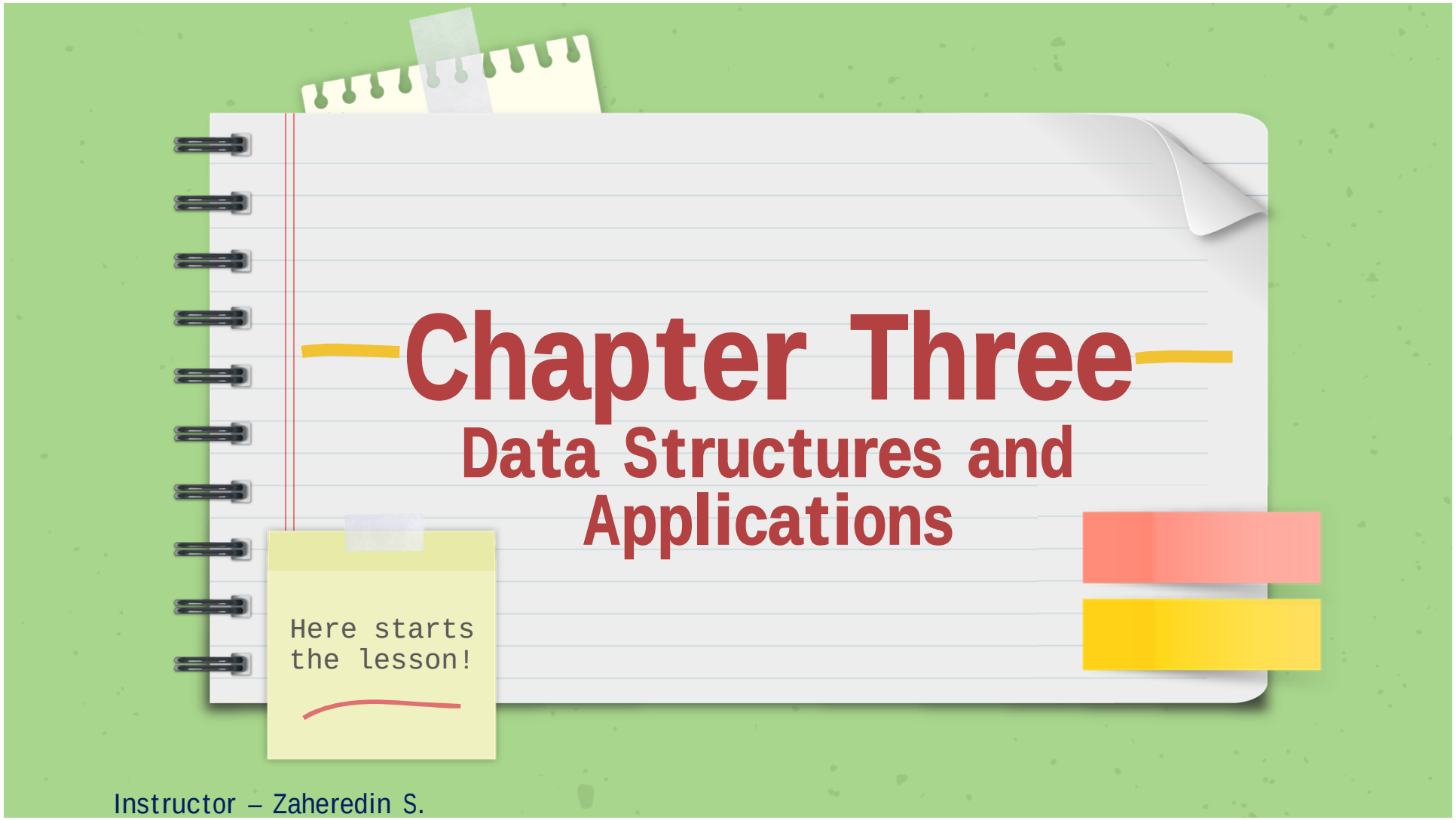


Instructor – Zaheredin S.

Chapter Three Contents!



3.1 An overview of Data Structure

Chapter Three Contents!



3.2 Linear Data Structure

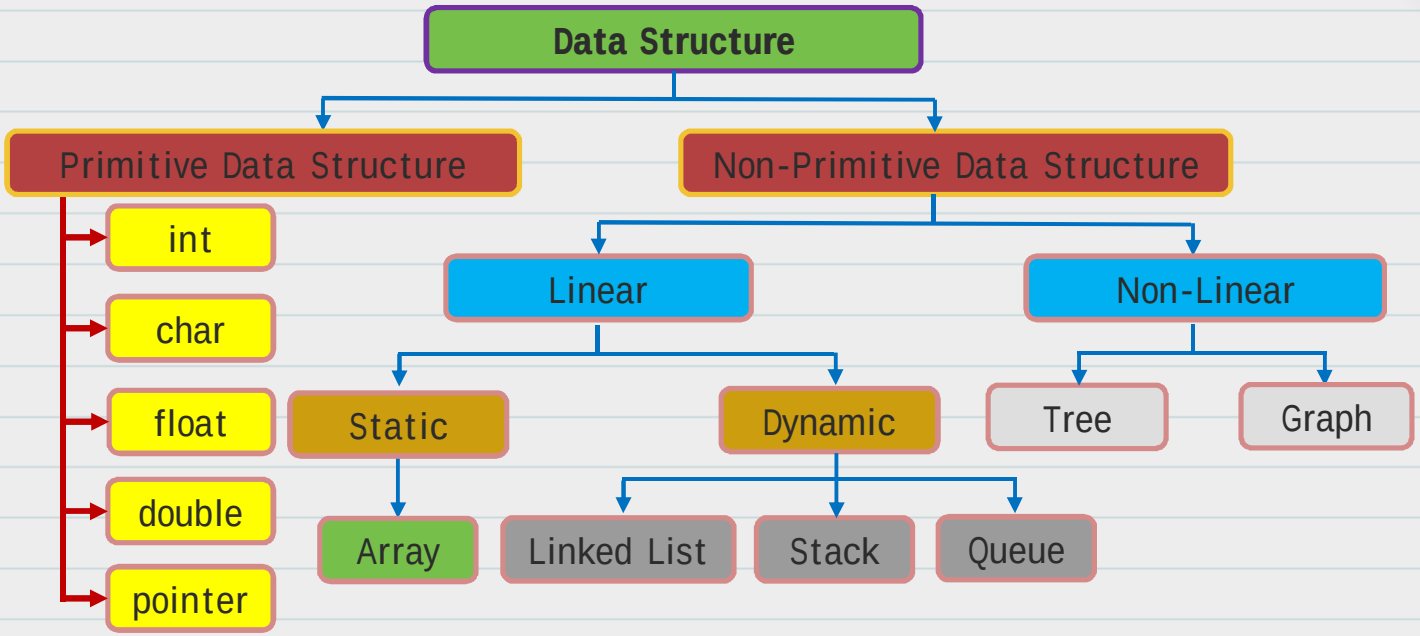
3.1 – Overview of Data Structure

What is "Data Structure"?

- **Data Structure** is a way to store and organize the data in memory so that it can be used efficiently.
- It is not a programming language like C, C++, java, etc. But, it is a set of algorithms that we can use in any programming language to structure the data in the memory.
- To structure the data in memory, '**n**' number of algorithms were proposed, and all these algorithms are known as **Abstract data types**.

Cont...

Data Structure Classification



Cont...

i. Primitive Data structure

- The primitive data structures are primitive data types.
- The int, char, float, double, and pointer are the primitive data structures that can hold a single value.

ii. Non-Primitive Data structure

- The non-primitive data structure is divided into two types:
  - ❖ Linear data structure
  - ❖ Non-linear data structure

Cont...

a) Linear Data Structure

- The arrangement of data in a sequential manner is known as a linear data structure.
- In these data structures, one element is connected to only one another element in a linear form.
- **Examples** - Arrays, Linked list, Stacks, and Queues.

b) None Linear Data Structure

- When one element is connected to the '**n**' number of elements known as a non-linear data structure.
- Here, the elements are arranged in a random manner.

Cont...

- The best example is trees and graphs.

Advantages of Data Structures

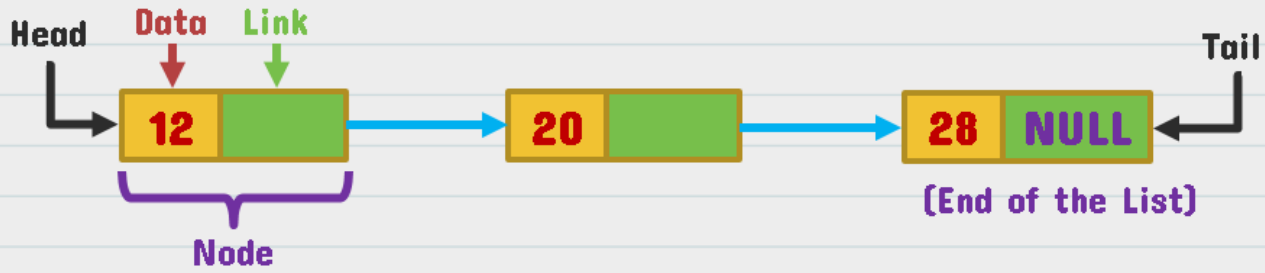
- **Efficiency** - Efficiency of a program depends upon the choice of data structures.
- **Reusability** - Data structures are reusable, i.e. multiple client programs can use the data structure.
- **Abstraction** - The client cannot see the internal working of the data structure, so it does not have to worry about the implementation part. The client can only see the interface.

3.2 – Linear Data Structures

a) Linked list

- **Linked list** is a linear data structure which is used to maintain a list in the memory.
- It is a collection of elements, but the elements are stored at non-contiguous memory locations.
- It can also be defined as the collection of the nodes in which one node is connected to another node.
- In linked list, a node consists of two parts, one is the **data part** and the second one is the **address part** as shown below figure:

Cont...



Cont...

Types of Linked List

I. Singly Linked list

- The singly linked list is a data structure that contains two parts, i.e., one is the **data part**, which stores actual information, and the other one is the **address part**, which contains the address of the next or the successor node. The address part in a node is also known as a pointer.
- In this list, only forward traversal is possible; we cannot traverse in the backward direction as it has only one link in the list.

Cont...

- In other words, we can say that each node contains only next pointer, therefore we can not traverse the list in the reverse direction.
- Each node has data and a pointer to the next node.



- The pointer that holds the address of the initial node is known as a head pointer.
- The last node of the list contains pointer to the null.

Cont...

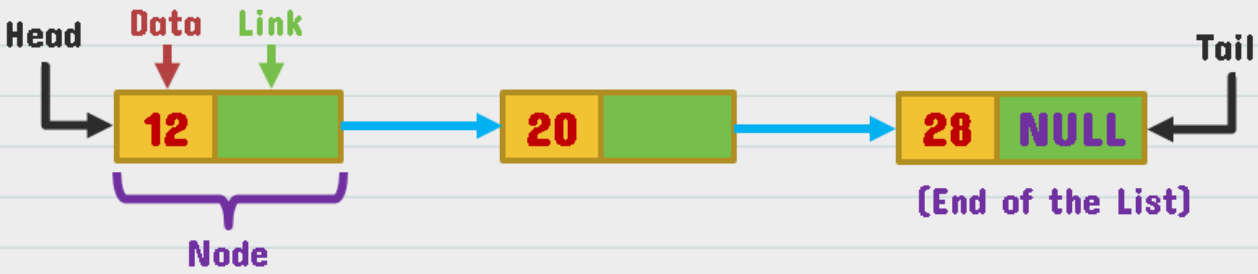


Figure:- Singly Linked List representation

Cont...

Operations on Singly Linked List

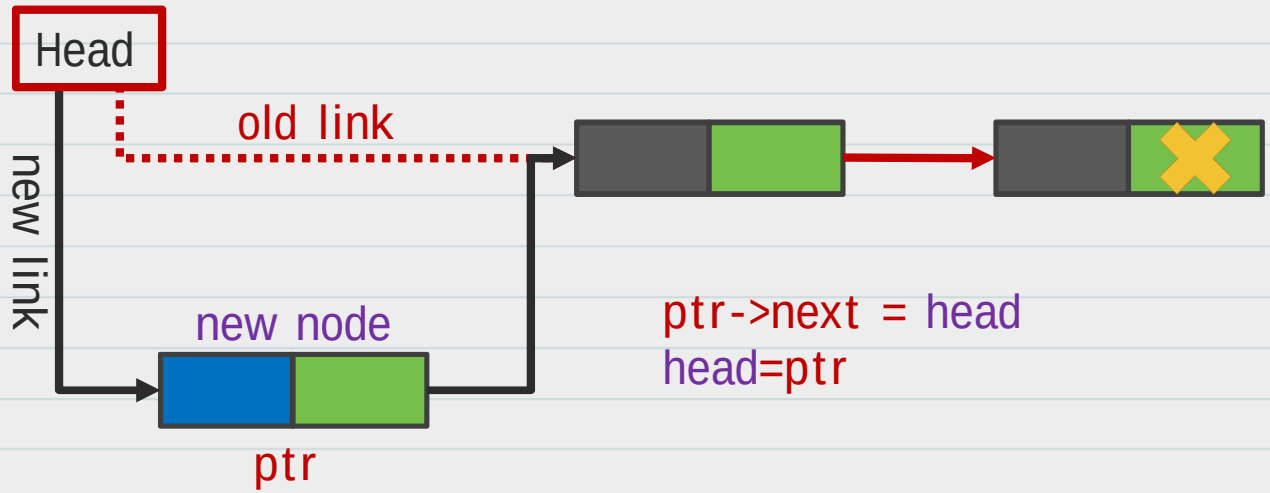
I. Insertion

The insertion into a singly linked list can be performed at different positions.

- **Insertion at beginning** – it involves inserting any element at the front of the list.
- **Insertion after specified node** - it involves insertion after the specified node of the linked list.
- **Insertion at the end** - it involves insertion at the last of the linked list.

Cont...

Insertion at beginning



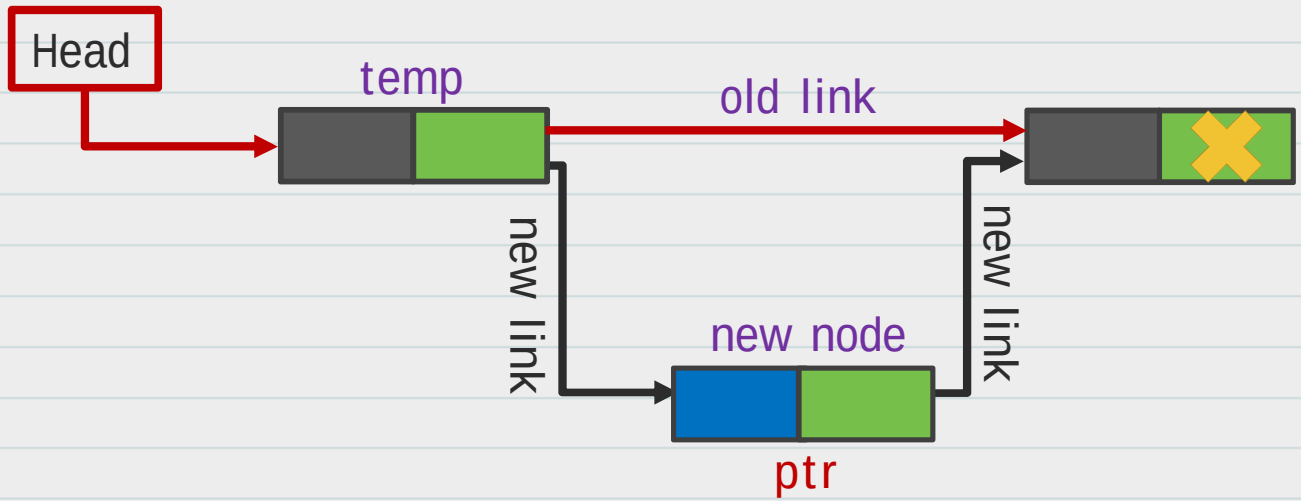
Cont...

Assume you want to insert node 19 at the beginning



Cont...

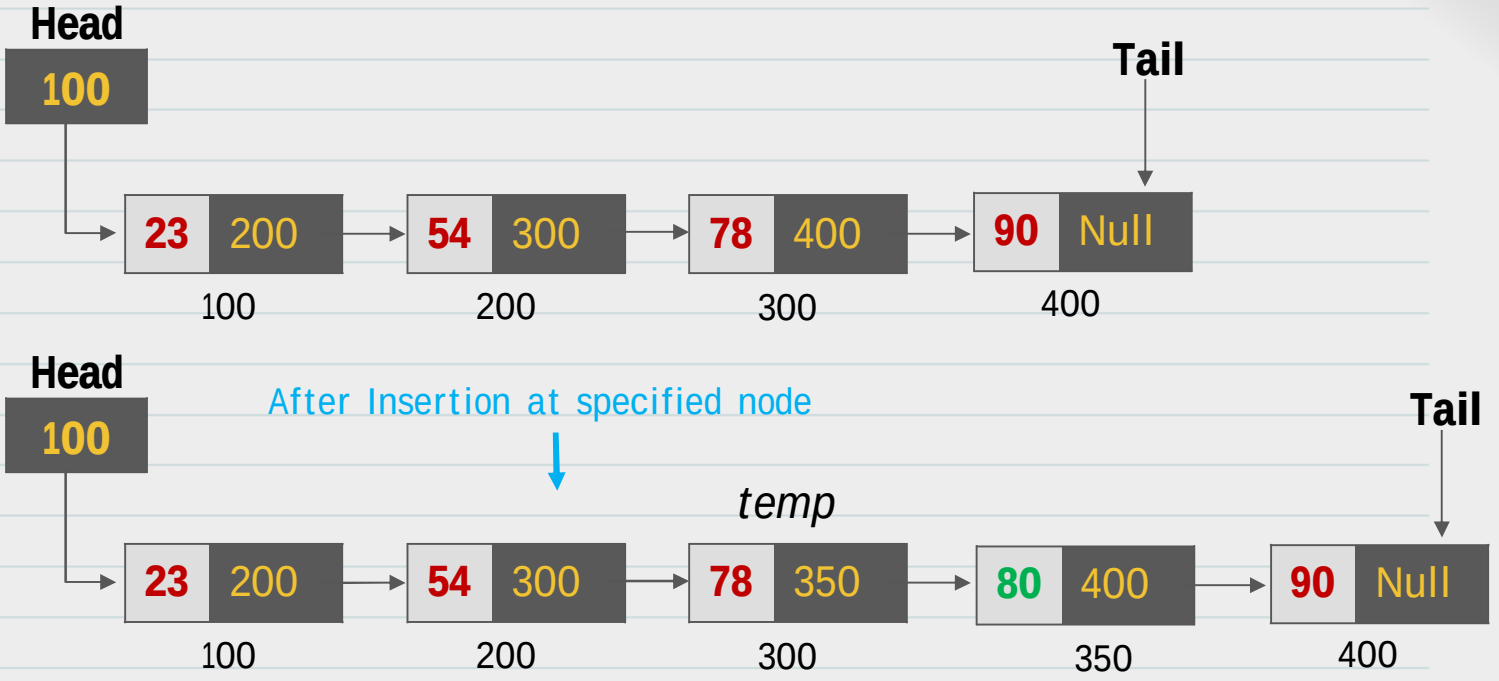
Insertion after specified node




```
ptr->next = temp->next
temp->next=ptr
```

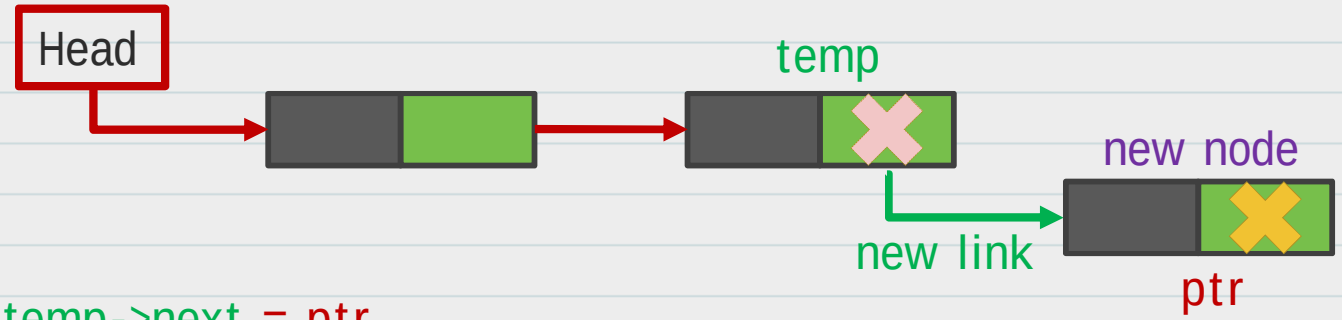
Cont...

Insert node 80 at the specified position 4 of the linked list.



Cont...

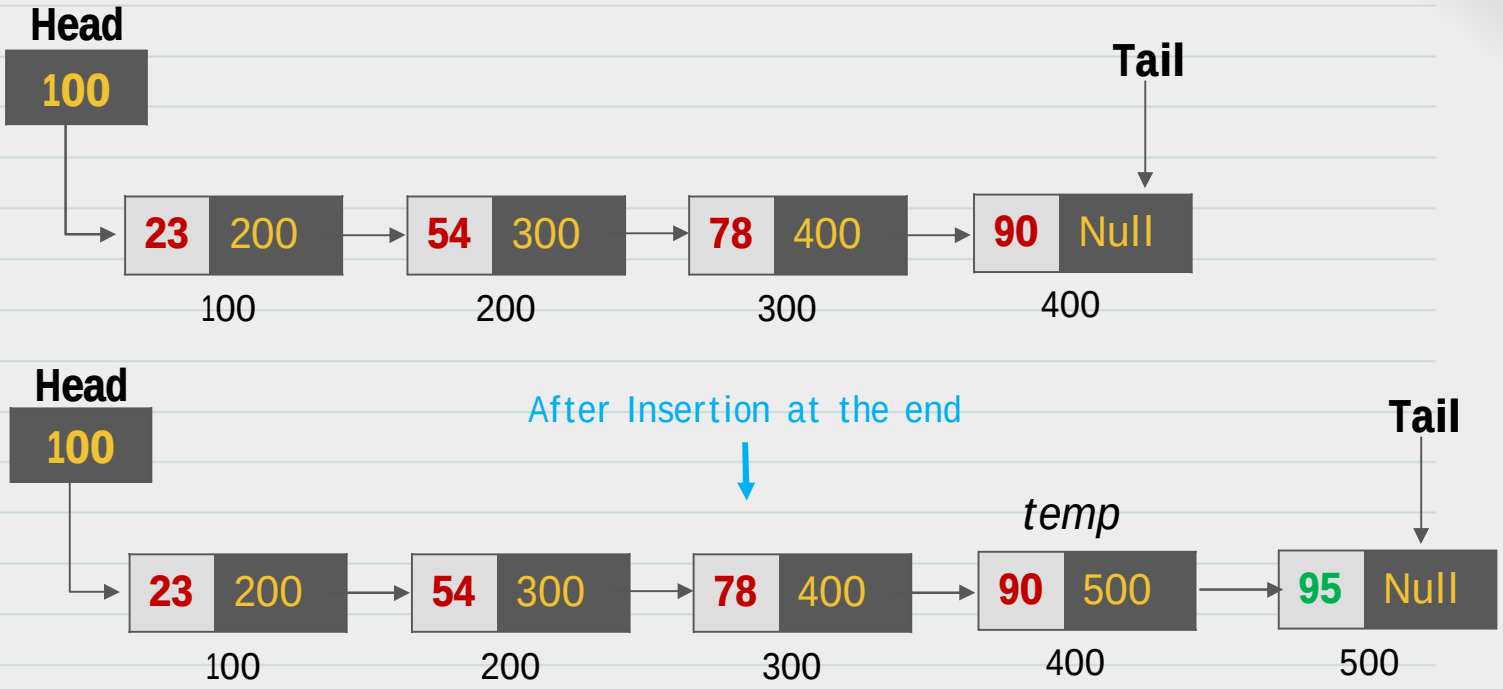
Insertion at the end



temp->next = ptr
ptr->next =NULL

Cont...

Assume you want to insert node 95 at the end.



Cont...

Operations on Singly Linked List

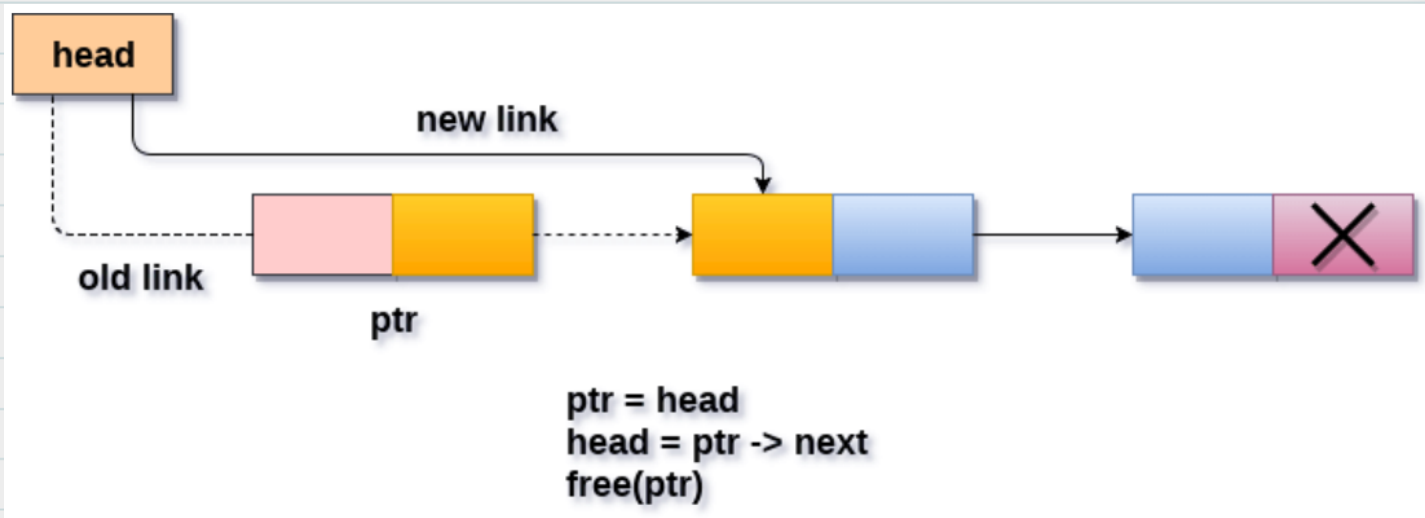
II. Deletion

The Deletion from a singly linked list can be performed at different positions.

- **Deletion at beginning** – it involves deletion of a node from the beginning of the list.
- **Deletion after specified node** - it involves deleting the node after the specified node in the list.
- **Deletion at the end** - it involves deleting the last node of the list.

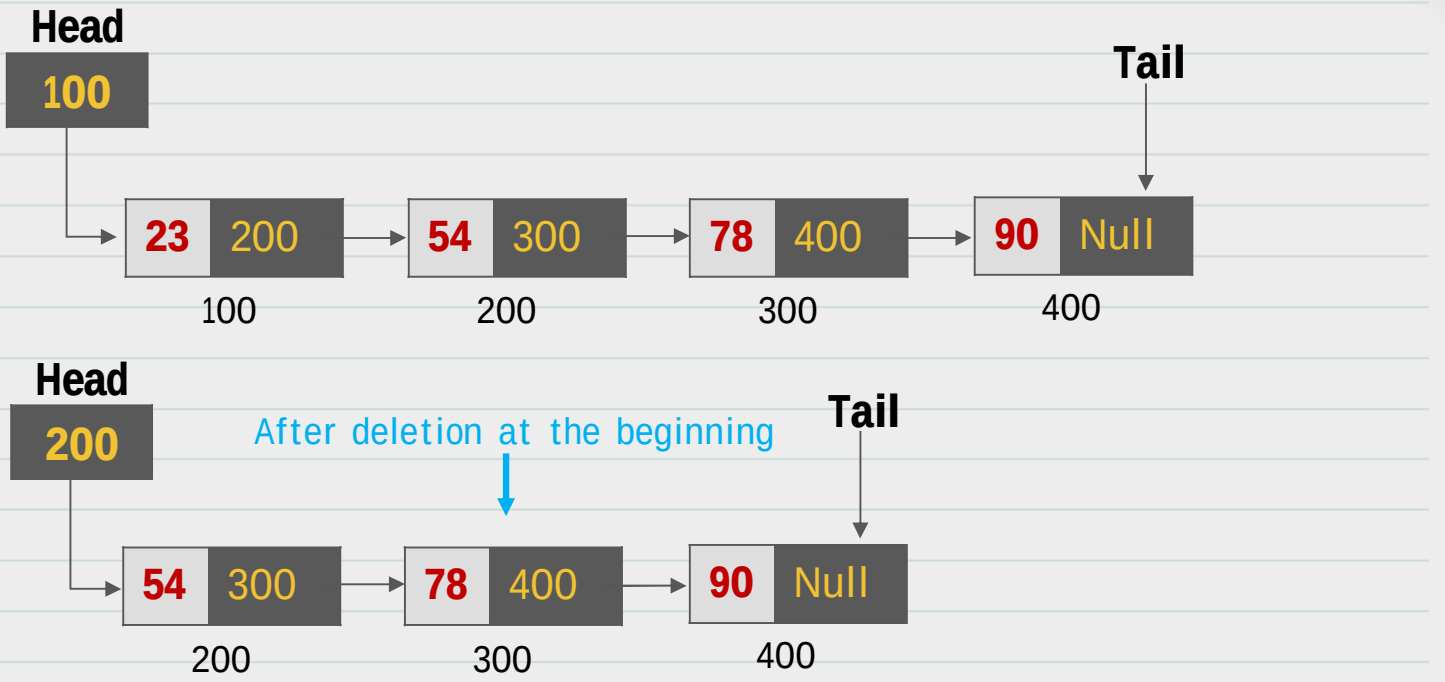
Cont...

Deletion at the beginning



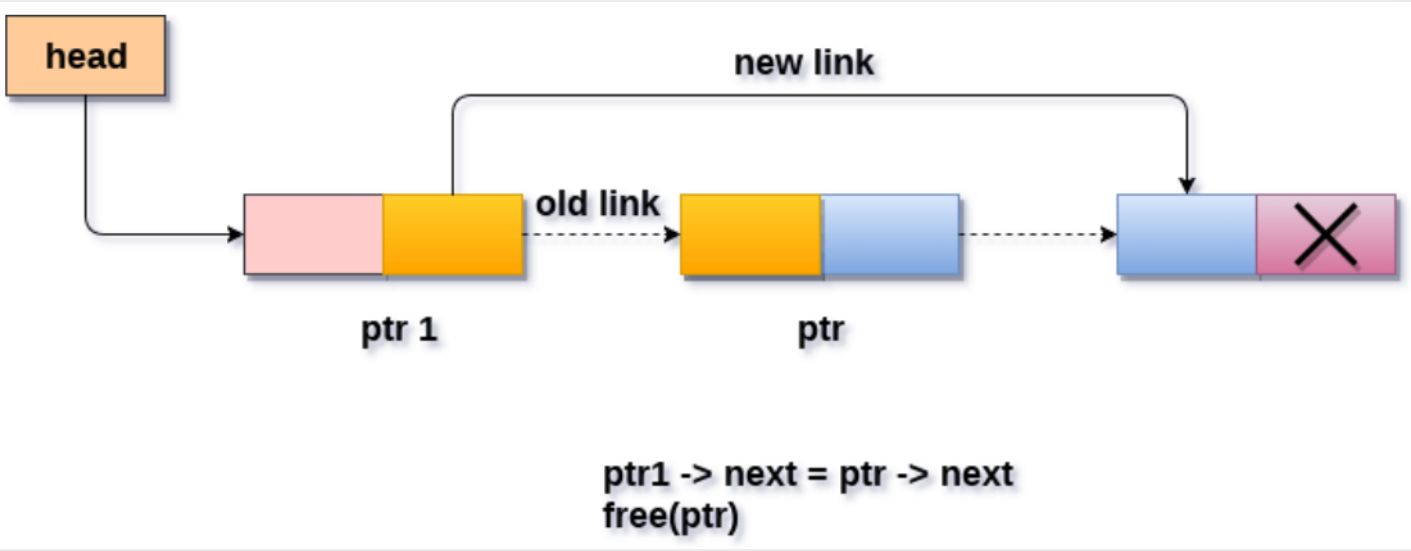
Cont...

Assume you want to delete node 23 at the beginning.



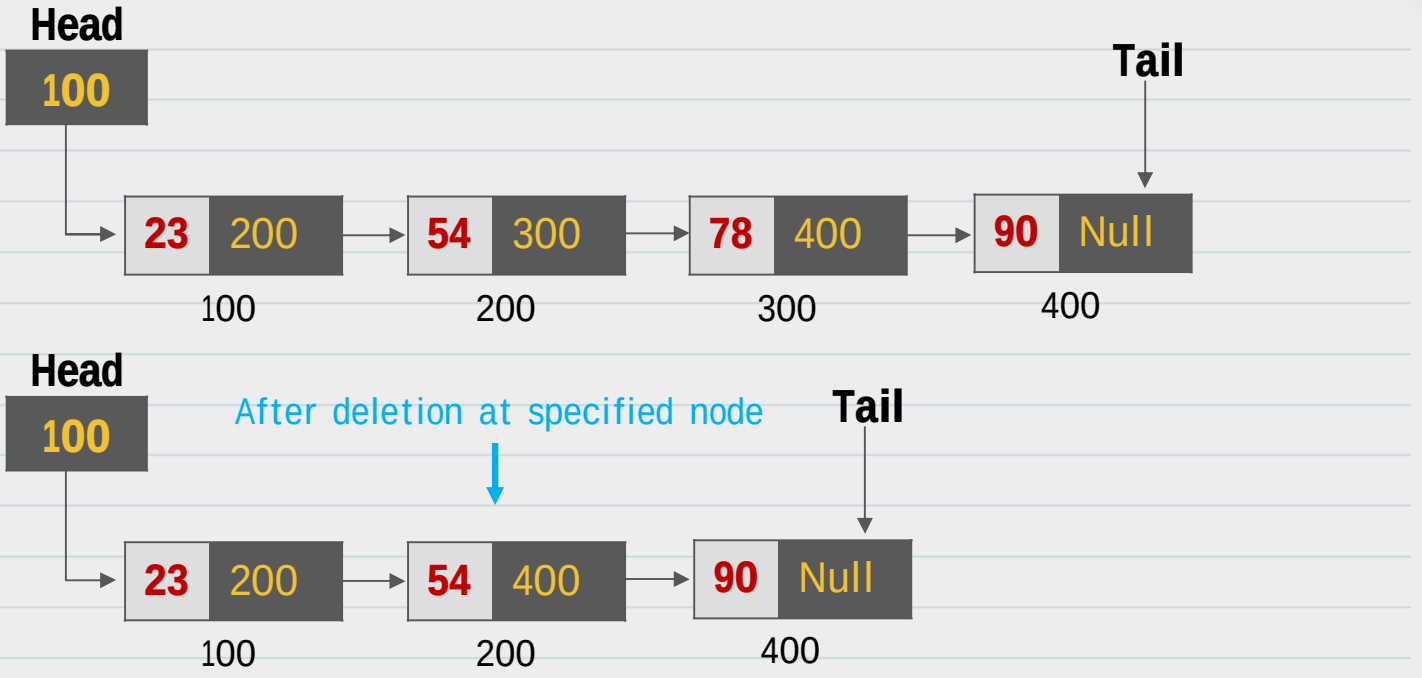
Cont...

Deletion after the specified node



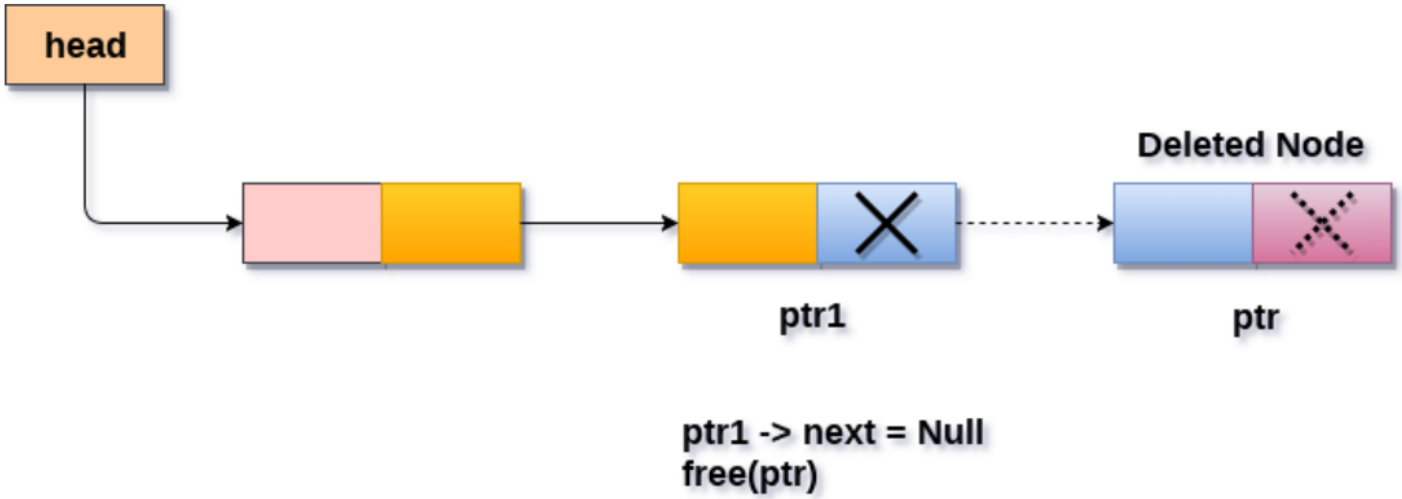
Cont...

Delete at the specified position of 3 (disconnect node 3).



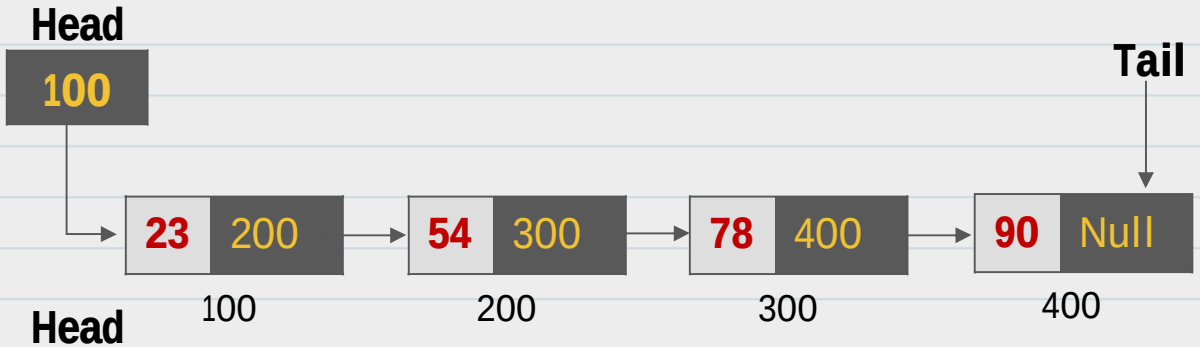
Cont...

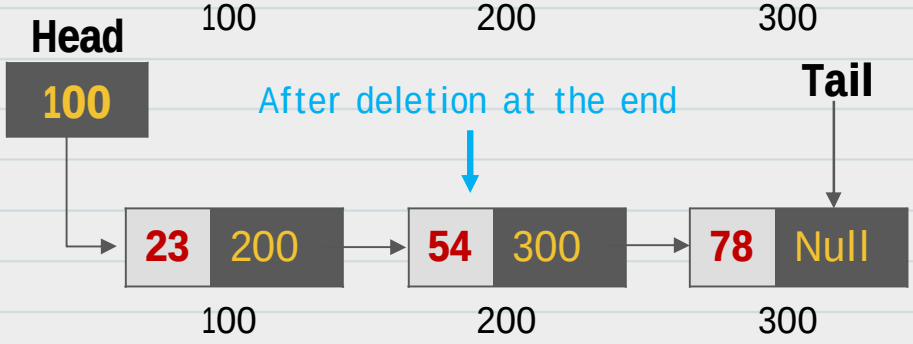
Deletion at the end



Cont...

Delete at the end, disconnect the Last node and the tail point at previous node





Cont...

Declaration of singly Linked List

Node is represented :as

```
struct node {
        int data;
        struct node *next;
 }
```

Cont...

Types of Linked List

II. Doubly Linked list

- Doubly linked list allows element two way traversal.
- The doubly linked list is a complex linked list to manage memory with previous and next pointer.
- We add a pointer to the previous node in a doubly-linked list.
- Thus, we can go in either direction: forward or backward.

Cont...

- In other words, it is a list that has three parts in a single node, includes one data part, a pointer to its previous node, and a pointer to the next node.



Node is represented as:

```
struct node {
    int data;
    struct node *next;
    struct node *prev;
}
```

Cont...

Operations on Doubly Linked List

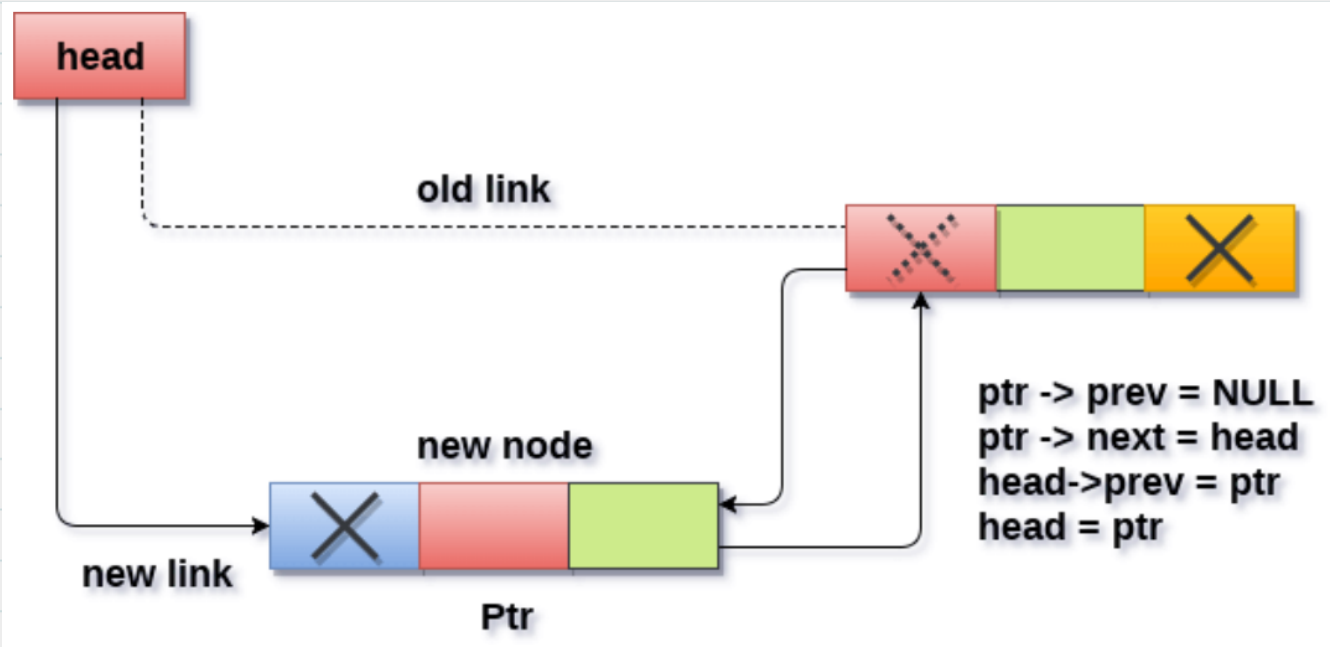
I. Insertion

The Insertion into a doubly linked list can be performed at different positions.

- **Insertion at beginning** – adding the node into the linked list at beginning.
- **Insertion after specified node** - adding the node into the linked list after the specified node.
- **Insertion at the end** - adding the node into the linked list at the end.

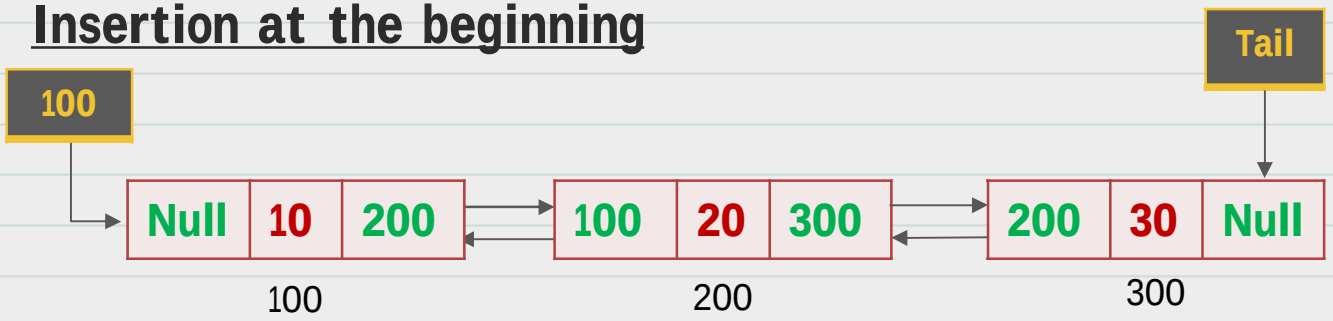
Cont...

Insertion at the beginning

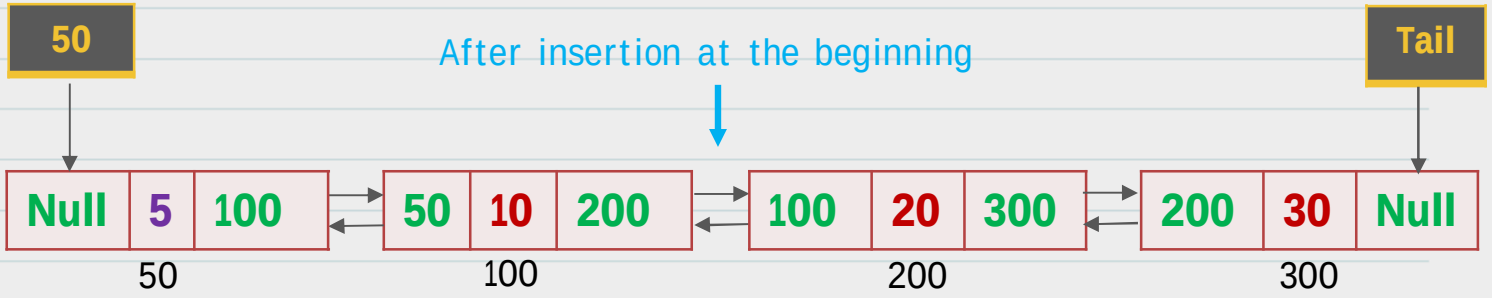


Cont...

Insertion at the beginning

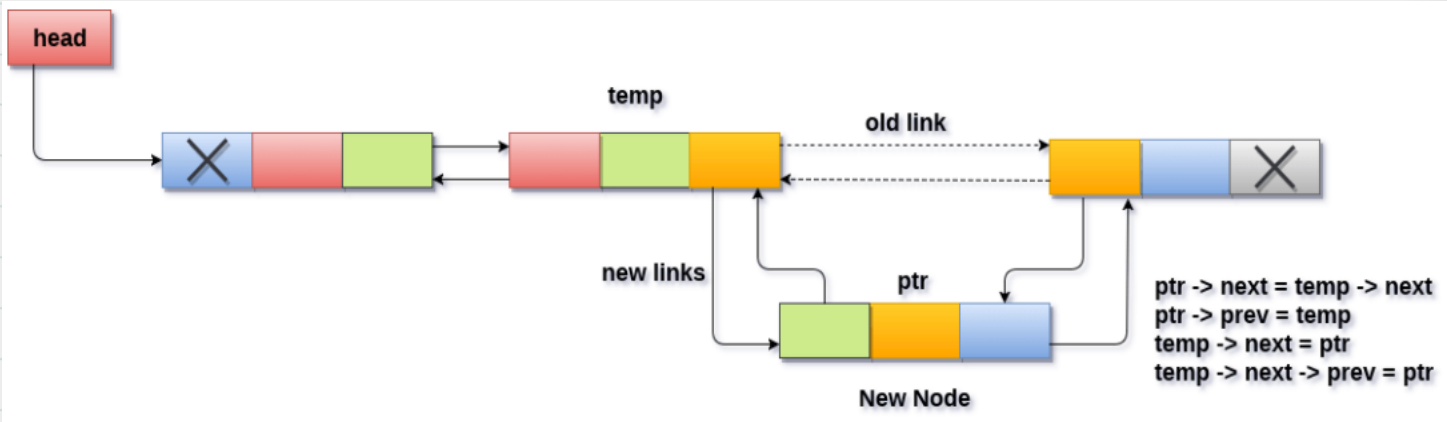


Insert node 5 at the Beginning doubly linked list



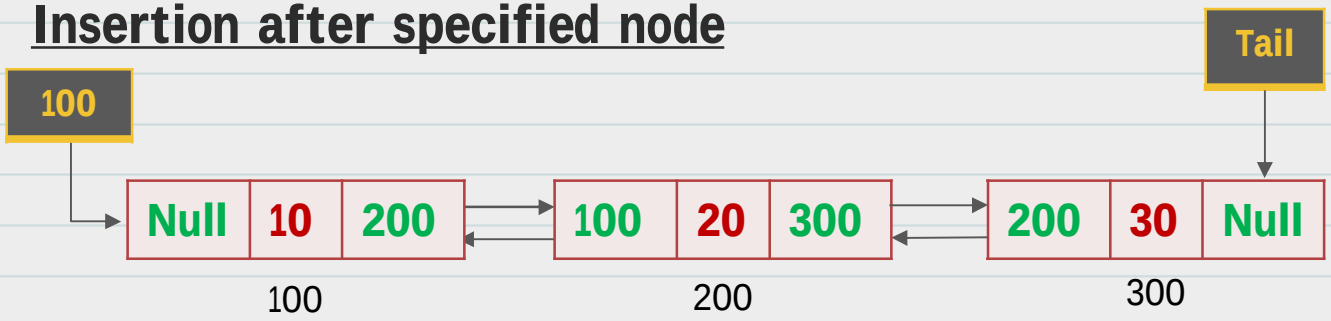
Cont...

Insertion after specified node

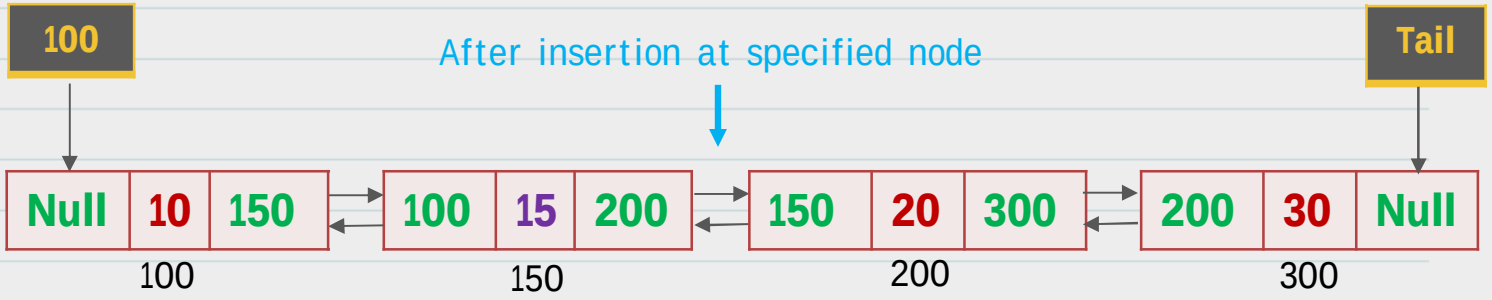


Cont...

Insertion after specified node

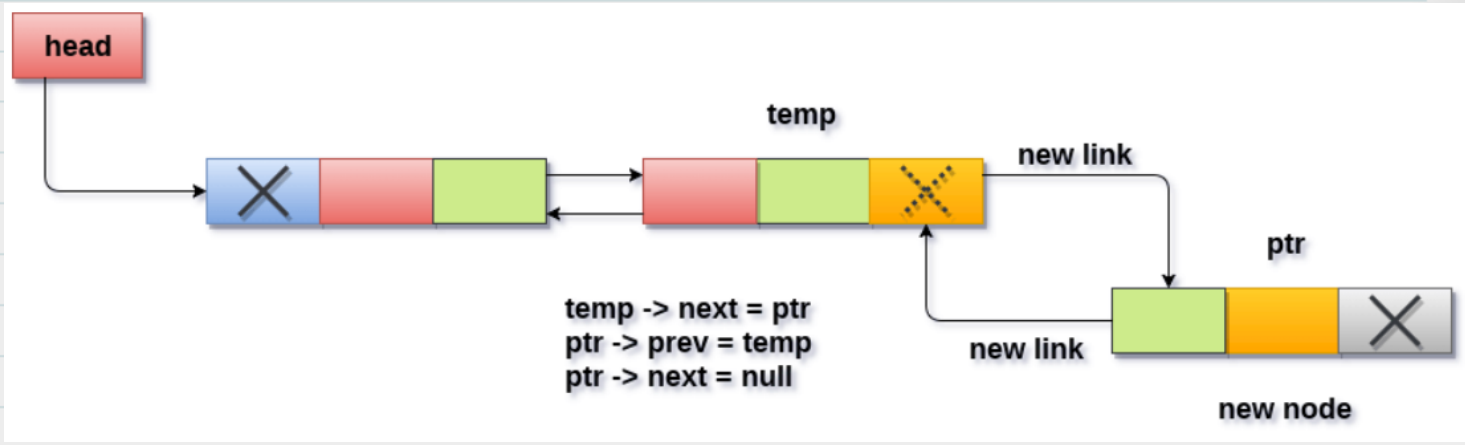


Insert node 15 at specified position 2 in doubly linked list



Cont...

Insertion at the end

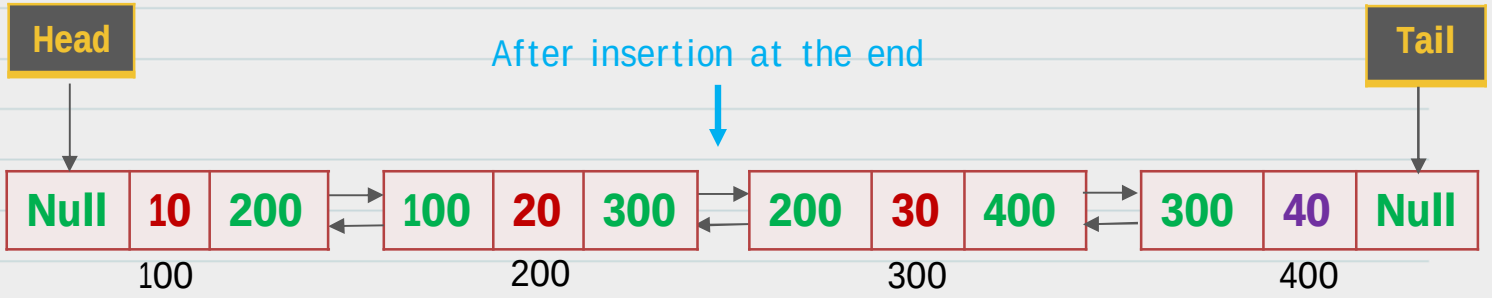


Cont...

Insertion at the end



Insert node 40 at the end of doubly linked list



Cont...

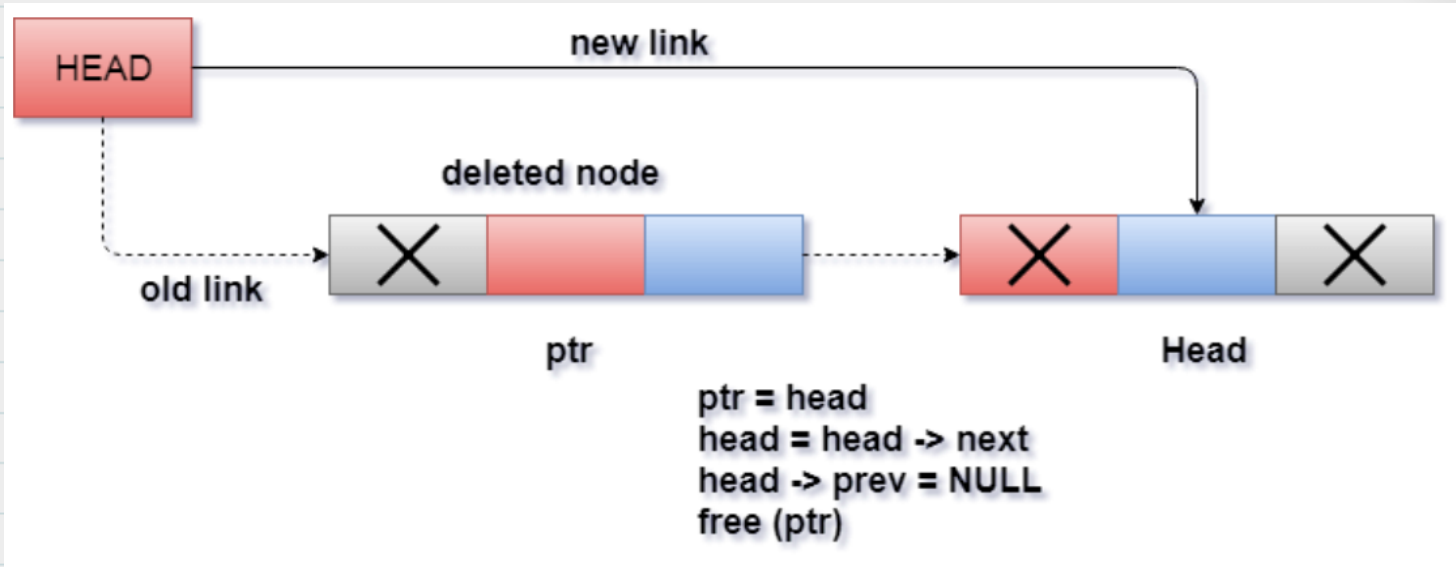
Operations on Doubly Linked List

II. Deletion

- Deletion at beginning – removing the node from beginning of the list
- Deletion of the node having a given data - removing the node which is present just after the node containing the given data.
- Deletion at the end - removing the node from end of the list.

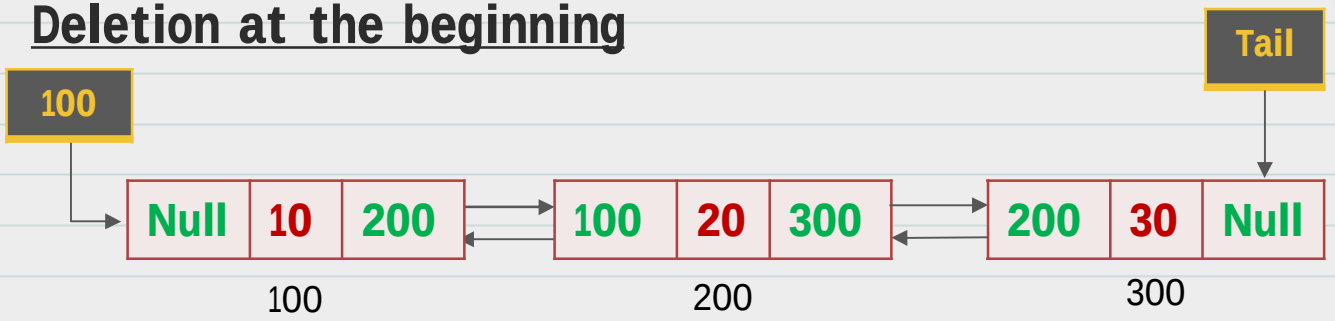
Cont...

Deletion at the beginning

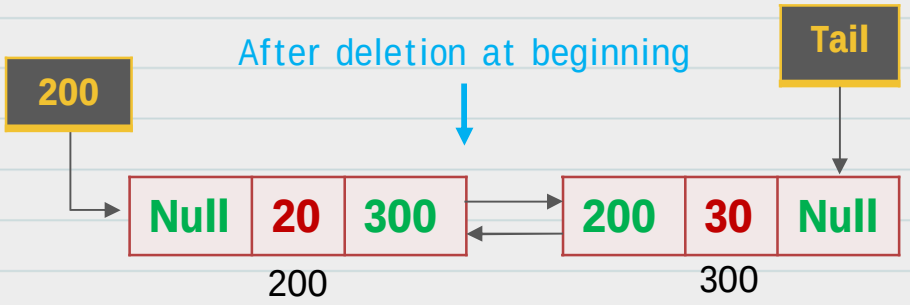


Cont...

Deletion at the beginning

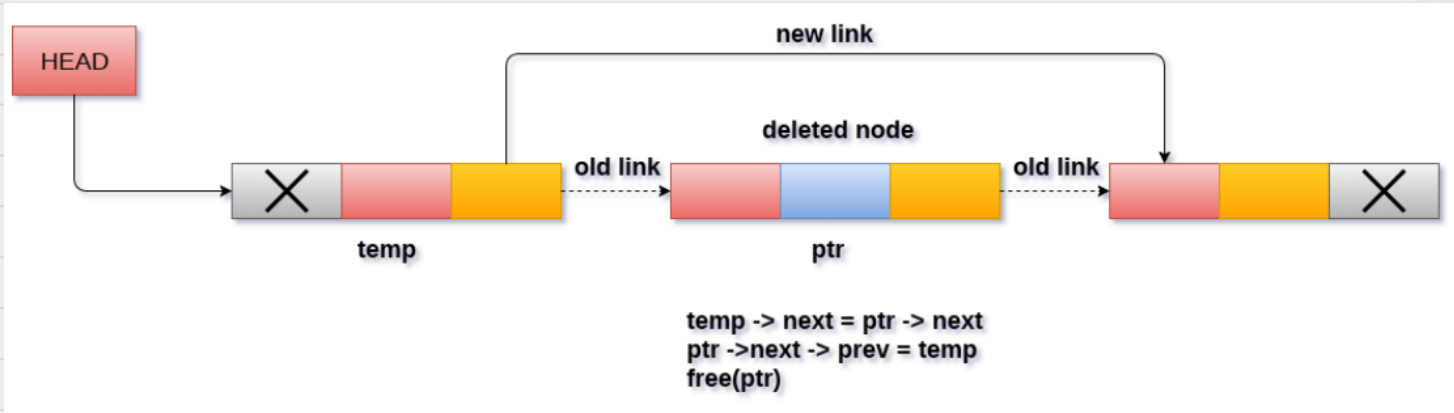


Disconnect the First node and the Head point at Second node.



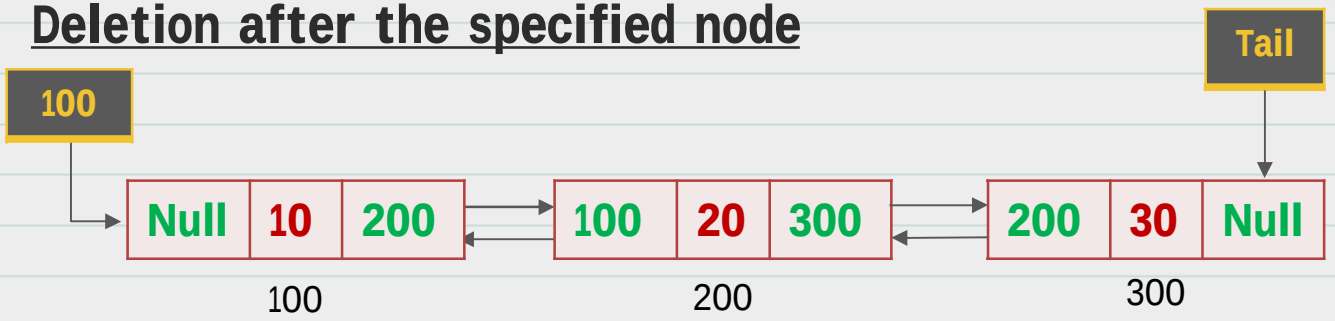
Cont...

Deletion after the specified node

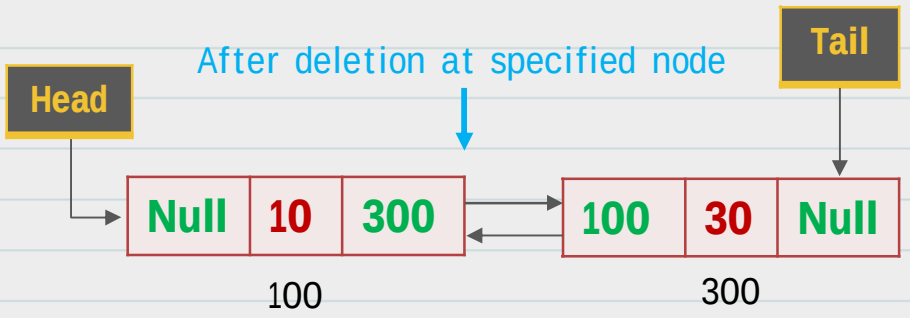


Cont...

Deletion after the specified node

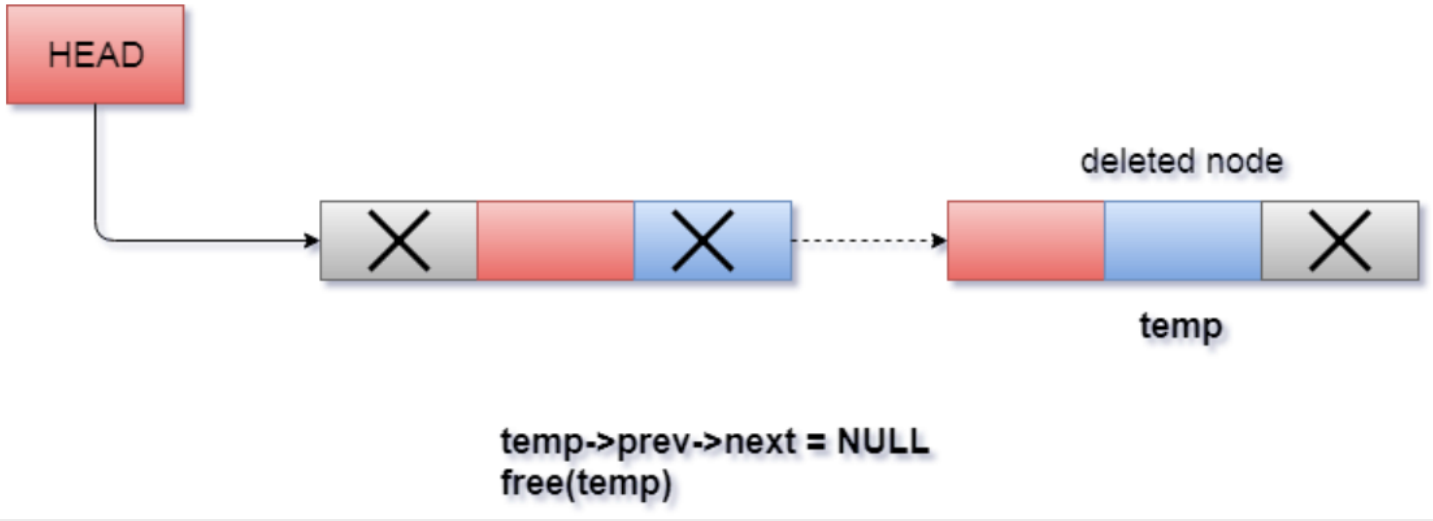


Delete at Specified position 2 in Doubly Linked List



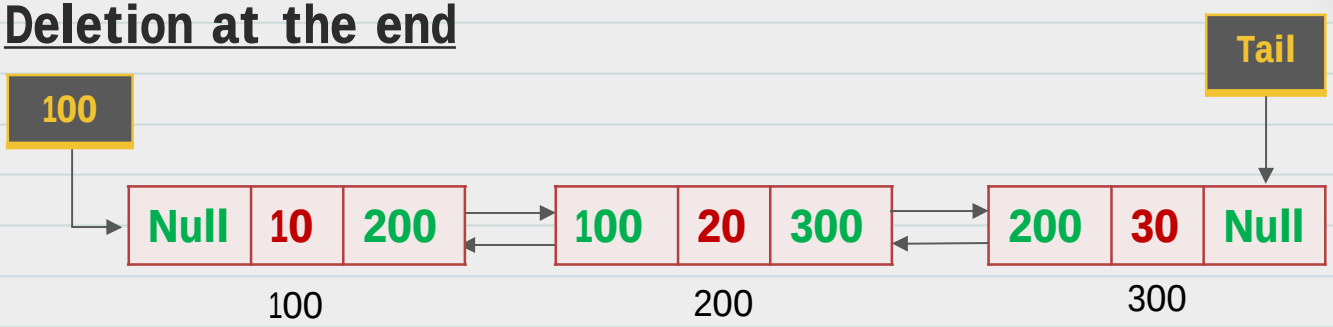
Cont...

Deletion at the end

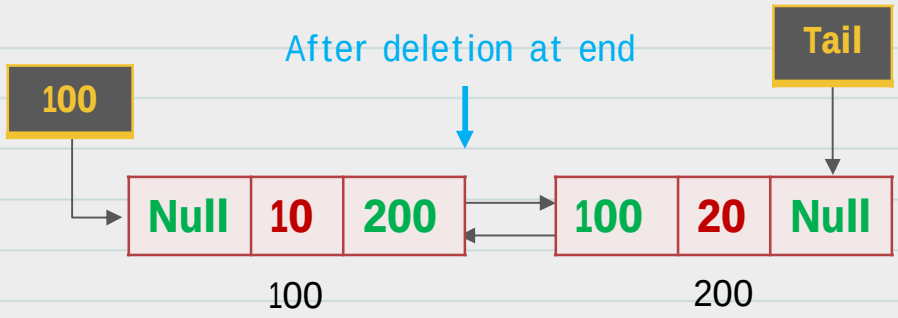


Cont...

Deletion at the end



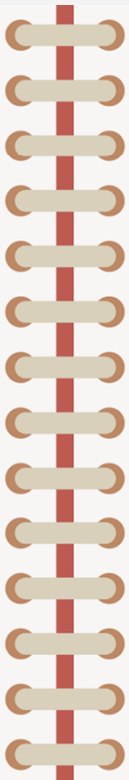
Delete the End Node of doubly linked list



Singly Linked List Vs Doubly Linked List

Singly Linked List

- It is a function that contains data and the next node only.
- It is a simple linked list to traverse one way from the first node to the next node.
- It contains two parts, such as memory and pointer, but the last pointer becomes null.

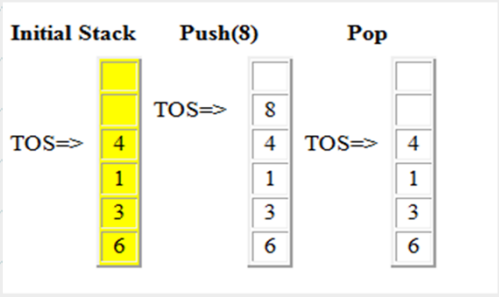


Doubly Linked List

- It is a function that contains data, previous node & next node.
- It is complex linked list to traverse both ways from one node to another & vice versa.
- It contains three parts such as a previous pointer, memory node, and next pointer but the initial and last pointer becomes null.

3.3 - Stacks

- **Stack** is a linear and an ordered list in which, insertion and deletion can be performed only at one end that is called top.
- It follows the LIFO (Last-In-First-Out) principle, i.e. the element which is inserted first in the stack, will be deleted last from the stack.
- The operations of insertion and deletion are called PUSH and POP
  - **Push()** - put item onto stack
  - **Pop()** - get item from stack



Cont...

Other stack operations

- **clear()** – remove all elements from the stack.
- **topEl()** – return the topmost element in the stack without removing it.
- **isEmpty()** - it determines whether the stack is empty or not.
- **isFull()** - it determines whether the stack is full or not.
- **display()** - it prints all elements available in the stack.

Cont...

PUSH operation

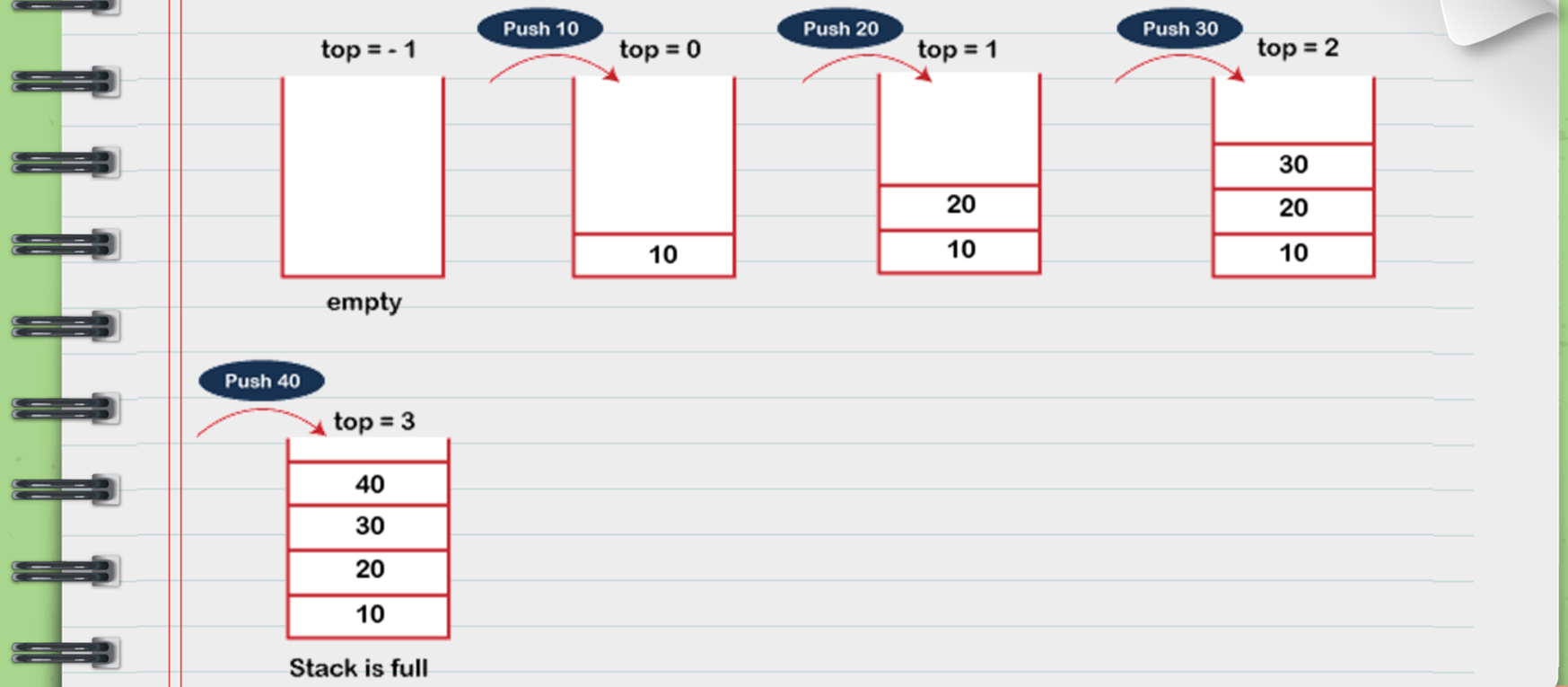
The steps involved in the PUSH operation is given below:

- Before inserting an element in a stack, we check whether the stack is full.
- If we try to insert the element in a stack, and the stack is full, then the overflow condition occurs.
- When we initialize a stack, we set the value of top as -1 to check that the stack is empty.

Cont...

- When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., top=top + 1, and the element will be placed at the new position of the top.
- The elements will be inserted until we reach the maximum size of the stack.

Cont...


top = - 1
empty
Push 10
top = 0
10
Push 20
top = 1
20
10
Push 30
top = 2
30
20
10
Push 40
top = 3
40
30
20
10
Stack is full

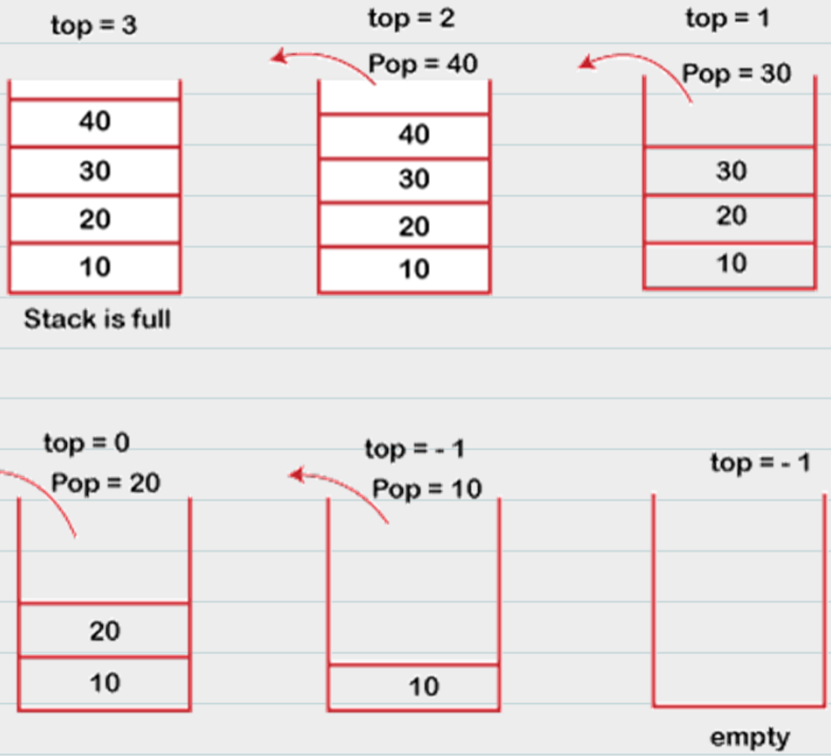
Cont...

POP operation

The steps involved in the POP operation is given below:

- Before deleting the element from the stack, we check whether the stack is empty.
- If we try to delete the element from the empty stack, then the underflow condition occurs.
- If the stack is not empty, we first access the element which is pointed by the top
- Once the pop operation is performed, the top is decremented by 1, i.e., top=top - 1.

Cont...


top = 3
40
30
20
10
Stack is full
top = 2
Pop = 40
40
30
20
10
top = 1
Pop = 30
30
20
10
top = 0
Pop = 20
20
10
top = - 1
Pop = 10
10
top = - 1
empty

3.4 - Queues

- A **queue** can be defined as a linear and an ordered list which enables insert operations to be performed at one end called **REAR** and delete operations to be performed at another end called **FRONT**.
- **Queue** is opened at both end therefore it follows First-In-First-Out (**FIFO**) methodology for storing the data items.
- **e.g.** people waiting in line for a rail ticket form a queue



Cont...

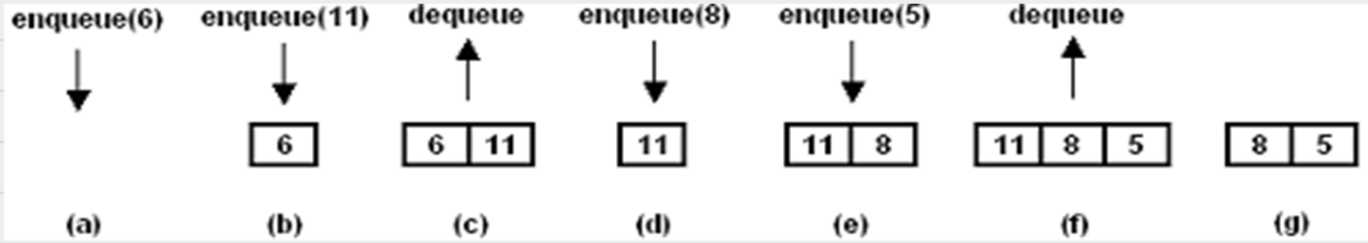
Queue operations

- **clear()** – remove all elements from the queue.
- **isEmpty()** – Check to see if there are elements in the queue.
- **enqueue(el)** – used to insert the element at the rear end of the queue.
- **dequeue()** – performs the deletion from the front-end of the queue.
- **firstEl()** – return the first element in the queue without removing it.

Cont...

A series of enqueue and dequeue operations is shown in Figure.

- This time, the new data elements are added to one end of the list (the right) and elements are removed from the other end (the left).
- For example, after enqueueing 6 & 11, the first dequeue operation (Figure c) removes the data element 6.



Cont...

- Similarly, at the dequeue (in Figure f) it is the 11 element that is removed, as this is at the head of the queue.


END
Thank You!